

# SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

## ASSESSMENT TOOL 1 – Constraint-Based Problem Solving

|                  |                                                                                                                                                                       |               |                                                      |
|------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------|------------------------------------------------------|
| Course Code:     | CSA06                                                                                                                                                                 | Course Title: | Design and Analysis of Algorithms                    |
| CO Assessed:     | CO5– Naturalization (BL5,BL6)                                                                                                                                         | Assessment:   | Assessment Tool 1 – Constraint Based Problem Solving |
| Weightage:       | 30% of CIA                                                                                                                                                            | Total Marks:  | 100                                                  |
| CO5 Statement:   | <i>Analyze computational problems using backtracking techniques and complexity classes such as P, NP, NP-Hard, and NP-Complete for combinatorial problem solving.</i> |               |                                                      |
| Student Name:    | s.sri mathi                                                                                                                                                           | Reg. No.:     | 192424095                                            |
| Section / Batch: | Slot b                                                                                                                                                                | Date:         | 27.08.2026                                           |

### Instructions to Students

- Answer the following question. Total Marks: 100.
- Apply backtracking techniques systematically for combinatorial problem solving.
- Clearly classify problems under P, NP, NP-Hard, and NP-Complete categories wherever required.
- Provide proper justification, state-space representation, and complexity analysis for all solutions.
- Neat presentation and logical explanation are mandatory.

| Question Type                    | Focus Area                                                                                                       | Questions |
|----------------------------------|------------------------------------------------------------------------------------------------------------------|-----------|
| Constraint Based Problem Solving | Analyze combinatorial problems using backtracking and complexity classes such as P, NP, NP-Hard, and NPComplete. | Q1        |

### SECTION A – CONSTRAINT-BASED PROBLEM SOLVING

**Code of Conduct**

I s.sri mathi (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: \_\_\_\_\_

**RUBRICS FOR CALCULATION**

| Criteria                | Wt. | Level 4 – Exemplary                                                            | Level 3 – Proficient                                  | Level 2 – Developing                          | Level 1 – Beginning                      |
|-------------------------|-----|--------------------------------------------------------------------------------|-------------------------------------------------------|-----------------------------------------------|------------------------------------------|
| Problem Understanding   | 25% | Clearly defines the problem and identifies all relevant constraints accurately | Identifies most constraints with minor gaps           | Partial understanding; misses key constraints | Fails to identify constraints correctly  |
| Constraint Analysis     | 25% | Analyzes constraints effectively and prioritizes them logically                | Provides reasonable analysis with some prioritization | Limited analysis; weak prioritization         | No meaningful analysis or prioritization |
| Solution Development    | 25% | Develops optimal and feasible solutions satisfying all constraints             | Develops workable solutions with minor limitations    | Solutions partially satisfy constraints       | Solutions do not meet constraints        |
| Application of Concepts | 15% | Effectively applies appropriate concepts, methods, or tools                    | Applies concepts with minor errors                    | Limited application; requires guidance        | Unable to apply relevant concepts        |
| Justification           | 10% | Provides strong justification for decisions with logical reasoning             | Provides justification with some clarity              | Weak or unclear justification                 | No justification provided                |

**Marks Evaluation:**

| Criteria                | Wt. | Level 4 – Exemplary | Level 3 – Proficient | Level 2 – Developing | Level 1 – Beginning |
|-------------------------|-----|---------------------|----------------------|----------------------|---------------------|
| Problem Understanding   | 25% |                     |                      |                      |                     |
| Constraint Analysis     | 25% |                     |                      |                      |                     |
| Solution Development    | 25% |                     |                      |                      |                     |
| Application of Concepts | 15% |                     |                      |                      |                     |
| Justification           | 10% |                     |                      |                      |                     |

**Total Marks (100):**

| Faculty In-charge | Course Coordinator | Head of Department |
|-------------------|--------------------|--------------------|
| Name: _____       | Name: _____        | Name: _____        |
| Signature: _____  | Signature: _____   | Signature: _____   |
| Date: _____       | Date: _____        | Date: _____        |

# CO5-AT1

## Constraint-Based Problem Solving

---

**NAME:** S.SRI MATHI

**REG.NO:** 192424095

**COURSE CODE:** CSA0603

**COURSE NAME:** DESIGN AND ANALYSIS OF ALGORITHM

**DATE:** 27.08.2026

---

### Q35. Graph Bandwidth Minimization (NP-Complete Problem)

Arrange graph vertices linearly such that the maximum edge length is minimized. Clearly define constraints related to vertex ordering and edge distance. Analyze how constraints impact feasible arrangements. Develop a backtracking-based solution. Apply pruning to reduce search space. Justify correctness and discuss NP-Complete complexity.

## 1. Problem Statement

**Graph Bandwidth Minimization** is a combinatorial optimization problem in which the vertices of a graph are arranged in a linear order so that the **maximum distance between the positions of adjacent vertices is minimized**.

Given an undirected graph:

$$G=(V,E)$$

where:

- $V$  = set of vertices
- $E$  = set of edges
- $n=|V|$  = number of vertices

we need to find a permutation of the vertices:

$$\pi=(v_1,v_2,\dots,v_n)$$

such that the **bandwidth** is minimum.

For every edge  $(u,v) \in E(u,v)$  in  $E$ , its distance is:

$$d(u,v) = |\text{pos}(u) - \text{pos}(v)|$$

The bandwidth of an ordering is:

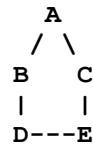
$$B(\pi) = \max_{(u,v) \in E} |\text{pos}(u) - \text{pos}(v)|$$

Therefore, the objective is:

$$\Pi \min B(\pi)$$

## 2. Example

Consider the graph:



One possible ordering is:

|                  |   |   |   |   |   |
|------------------|---|---|---|---|---|
| <b>Position:</b> | 1 | 2 | 3 | 4 | 5 |
| <b>Vertex:</b>   | A | B | C | D | E |

For the edge  $(A,C)$ :

$$|1-3| = 2$$

For the edge  $(B,D)$ :

$$|2-4| = 2$$

For the edge  $(C,E)$ :

$$|3-5| = 2$$

For the edge  $(D,E)$ :

$$|4-5| = 1$$

Hence:

$$B(\pi) = 2$$

The goal is to find an ordering with the **smallest possible maximum edge distance**.

### 3. Constraints

The solution must satisfy the following constraints.

| Constraint        | Description                                           |
|-------------------|-------------------------------------------------------|
| Unique vertices   | Every vertex must appear exactly once                 |
| Linear ordering   | Each vertex receives exactly one position from 1 to n |
| Edge preservation | Every original edge must remain an edge               |
| Edge distance     | For edge (u,v), distance is (                         |
| Bandwidth bound   | Every edge distance must be $\leq B$                  |
| Optimization      | Minimize the maximum edge distance                    |

#### **Important constraint**

For a proposed bandwidth  $B$ :

$$|\text{pos}(u) - \text{pos}(v)| \leq B$$

must hold for **every edge**  $(u,v)$ .

### 4. How Constraints Affect Feasible Arrangements

The constraints greatly reduce the number of possible vertex arrangements.

Without constraints,  $n$  vertices can be arranged in:

$$n!$$

different ways.

For example, with 5 vertices:

$$5! = 120$$

possible arrangements exist.

However, if a bandwidth bound  $B=2$  is imposed, many of these 120 arrangements become invalid because some connected vertices may be more than two positions apart.

#### **Example**

Suppose:

A ----- D

is an edge.

If the ordering is:

A B C D  
1 2 3 4

then:

$$|\text{pos}(\text{A}) - \text{pos}(\text{D})| = |1 - 4| = 3 \quad |\text{pos}(\text{A}) - \text{pos}(\text{D})| = |1 - 4| = 3$$

If the allowed bandwidth is  $B=2$ :

$$3 > 2 \quad 3 > 2$$

Therefore, this arrangement is **infeasible**.

This constraint-based elimination is what makes **backtracking with pruning** useful.

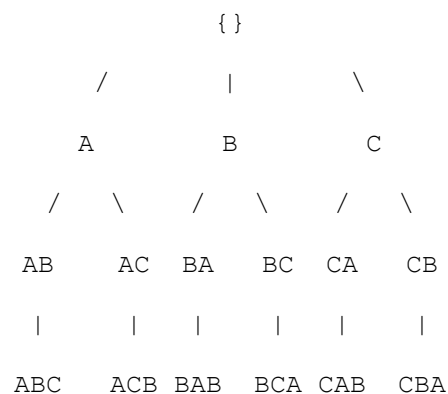
## 5. State-Space Representation

The state-space tree represents all possible vertex orderings.

For vertices:

**{A,B,C}**

the tree begins with an empty ordering.



Each level represents assigning one more vertex to a position.

### State representation

A state can be represented as:

$$S=(v_1,v_2,\dots,v_k)$$

where  $k$  vertices have already been placed.

The remaining vertices are candidates for the next position.

## 6. Backtracking Approach

The basic idea is:

1. Select an unused vertex.
2. Place it at the next position.
3. Check whether the partial ordering violates the bandwidth constraint.
4. If valid, recursively continue.
5. If invalid, **prune** that branch.
6. When all vertices are placed, calculate the bandwidth.
7. Keep the best ordering found.

## 7. Pruning Strategy

Pruning is essential because blindly checking all  $n!$  permutations becomes expensive.

Suppose the current partial ordering is:

| A | C | B |
|---|---|---|
| 1 | 2 | 3 |

and the next vertex DD is connected to AA.

If DD is placed at position 5:

| A | C | B | X | D |
|---|---|---|---|---|
| 1 | 2 | 3 | 4 | 5 |

then:

$$|\text{pos}(A) - \text{pos}(D)| = |1 - 5| = 4$$

If the current allowed bandwidth is 3:

$$4 > 3 > 3$$

Therefore, there is **no reason to explore this branch further**.



A C B  
|  
place D  
|  
invalid  
X  
PRUNE

### Main pruning condition

For every already placed vertex  $u$  connected to the newly placed vertex  $v$ :

$$|\text{pos}(u) - \text{pos}(v)| > B$$

means:

PRUNE

## 8. Backtracking Algorithm

### Algorithm: Graph Bandwidth Minimization

BACKTRACK(order, used, position)

1. If all vertices are placed:
  - Calculate bandwidth
  - Update best bandwidth and best ordering
  - Return
2. Choose an unused vertex  $v$ .
3. Place  $v$  at the current position.
4. Check all already placed neighbors  $u$  of  $v$ .
5. If
  - $|\text{position}(u) - \text{position}(v)| > \text{current\_best}$
  - then
    - Undo placement
    - Prune this branch
    - Return
6. Otherwise:
  - Recursively call BACKTRACK()

7. Remove  $v$  from the current position.
8. Try the next unused vertex.

## **9. Pseudocode**

```
bestBandwidth =  $\infty$ 
```

```
bestOrder = empty
```

```
BACKTRACK(order, used, pos):
```

```
    if pos > n:
```

```
        bandwidth = 0
```

```
    for each edge (u, v):
```

```
        distance = abs(position[u] - position[v])
```

```
        bandwidth = max(bandwidth, distance)
```

```
    if bandwidth < bestBandwidth:
```

```
        bestBandwidth = bandwidth
```

```
        bestOrder = order
```

```
    return
```

```
for each vertex v:
```

```
    if used[v] == false:
```

```
        position[v] = pos
```

```
        valid = true
```

```
        for each already placed neighbor u of v:
```

```
            distance = abs(position[u] - pos)
```

```
            if distance >= bestBandwidth:
```

```
                valid = false
```

```

        break

    if valid:

        used[v] = true
        order[pos] = v

        BACKTRACK(order, used, pos + 1)

    used[v] = false

    position[v] = 0

```

## 10. Improved Decision-Based Approach

A more effective method is to convert the optimization problem into a **decision problem**.

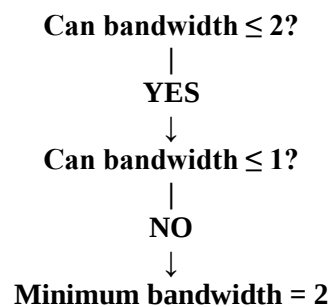
Instead of directly asking:

"What is the minimum bandwidth?"

ask:

"Can the vertices be arranged with bandwidth at most BB?"

For example:

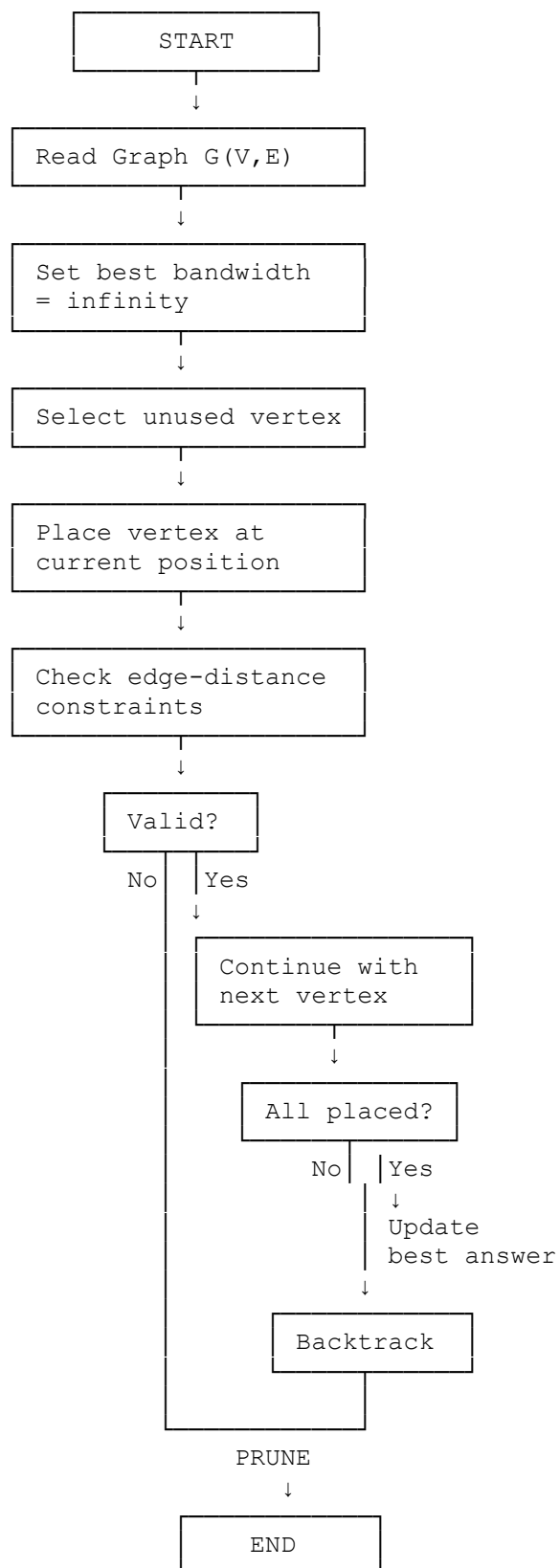


The backtracking algorithm tries to construct an ordering satisfying:

$$|\text{pos}(u) - \text{pos}(v)| \leq B$$

for every edge.

# 11. Flowchart



## **13. Correctness Justification**

The backtracking algorithm is correct because it systematically explores possible vertex orderings.

### **1. Completeness**

Every vertex is considered for every available position.

Therefore, every possible permutation can potentially be generated.

### **2. Constraint checking**

Whenever a vertex is placed, its distance from already placed adjacent vertices is checked.

If:

$$|\text{pos}(u) - \text{pos}(v)| > B$$

the arrangement cannot satisfy the bandwidth bound and is safely discarded.

### **3. Optimality**

Whenever a complete ordering is obtained, its bandwidth is calculated.

The algorithm stores the ordering having the smallest bandwidth.

Hence, after all feasible arrangements have been considered:

$$\text{bestBandwidth} = \text{minimum possible bandwidth}$$

### **Why pruning does not affect correctness**

A pruned branch already violates the required bandwidth constraint. Adding more vertices cannot change the positions of vertices already placed, so the violation cannot be repaired.

Therefore, pruning only removes **impossible solutions**, not valid optimal solutions.

## **14. Complexity Analysis**

There are  $n!$  possible permutations of  $n$  vertices.

In the worst case, the backtracking algorithm may examine all permutations.

Therefore, the time complexity is approximately:

$$O(n! \cdot (n + |E|))$$

depending on how the bandwidth and constraints are checked.

The space complexity is:

$$O(n+|E|)$$

for storing the graph, vertex positions, used array, ordering and recursion stack.

### With pruning

In practical cases, pruning can dramatically reduce the number of states explored. However, the **worst-case asymptotic complexity remains exponential/factorial**.

## 15. Complexity Classification

The problem is related to the classical **Bandwidth Problem**.

The **decision version** is:

Given a graph  $G$  and an integer  $B$ , does there exist a vertex ordering whose bandwidth is at most  $B$ ?

This decision problem is **NP-complete**.

| Complexity Class   | Meaning                                     | Relation to this problem                               |
|--------------------|---------------------------------------------|--------------------------------------------------------|
| <b>P</b>           | Solvable in polynomial time                 | General bandwidth minimization is not known to be in P |
| <b>NP</b>          | Solution can be verified in polynomial time | Yes                                                    |
| <b>NP-Hard</b>     | At least as hard as every problem in NP     | Optimization version is NP-Hard                        |
| <b>NP-Complete</b> | Both NP and NP-Hard                         | Decision version is NP-Complete                        |

### Why it belongs to NP

Given a proposed ordering, we can calculate the position of every vertex and check every edge:

$$|\text{pos}(u) - \text{pos}(v)| \leq B$$

This verification can be done in polynomial time.

Thus:

**Bandwidth Decision Problem**  $\in$  NP

The optimization version—finding the minimum bandwidth—is therefore **NP-Hard**.

## 17. Advantages of Backtracking

| Advantage         | Explanation                                  |
|-------------------|----------------------------------------------|
| Systematic        | Explores the solution space systematically   |
| Exact             | Finds the optimal bandwidth                  |
| Constraint-based  | Directly handles vertex-ordering constraints |
| Pruning           | Avoids impossible partial arrangements       |
| Easy to implement | Suitable for small graphs                    |
| Flexible          | Additional constraints can be incorporated   |

## 18. Limitations

The major limitation is computational cost.

For  $n$  vertices:

$$n!$$

possible arrangements exist.

For example:

| Vertices | Possible orderings |
|----------|--------------------|
| 4        | 24                 |
| 5        | 120                |
| 6        | 720                |
| 8        | 40,320             |
| 10       | 3,628,800          |
| 15       | 1,307,674,368,000  |

Therefore, backtracking is suitable mainly for **small or moderately sized graphs** unless strong pruning or specialized algorithms are used.

## 19. Summary

Graph Bandwidth Minimization arranges graph vertices linearly while minimizing the maximum distance between adjacent vertices.

The mathematical objective is:

$$\pi \min_{(u,v) \in E} |\text{pos}(u) - \text{pos}(v)|$$

A **backtracking algorithm** generates vertex orderings incrementally. Whenever a partial ordering violates the current bandwidth bound, that branch is immediately **pruned**.

The approach is correct because it explores all potentially feasible orderings and discards only arrangements that cannot produce a valid solution.

The decision version:

**“Is there an ordering with bandwidth  $\leq B$ ?”**

is **NP-Complete**, while the optimization version is **NP-Hard**. Thus, the problem demonstrates why backtracking is useful for exact solutions to difficult combinatorial problems, even though its worst-case running time is exponential/factorial.

## **Final classification**

**Bandwidth Decision Problem = NP-Complete**

This directly satisfies the required **backtracking + state-space + pruning + correctness + P/NP/NP-Hard/NP-Complete + complexity analysis** components of the question.